Programación con Objetos 3

Introducción a Ruby + Repaso sobre pruebas automatizadas y TDD

Autores:

- Facundo Javier Gelatti — javiergelatti@gmail.com

Este documento está bajo una licencia CC BY 4.0

Última modificación: 2026-03-28

Índice

Introducción a Ruby (y un repaso de otros temas)..... 2
El dominio: un juego de estrategia..... 2
Estructura del proyecto..... 2
Paso 1: la primera prueba..... 3
Elegir qué probar..... 3
El nombre de la prueba..... 4
Las tres partes de una prueba..... 5
Escribiendo la prueba..... 5
Analizando nuestras decisiones de diseño..... 7
Algunas características de Ruby..... 8
Todo es un objeto..... 8
Clases y “constructores”..... 9
Variables de instancia..... 10
Símbolos..... 12
nil..... 12
Paso 2: haciendo pasar la prueba..... 13
Paso 3: refactorizar..... 13
Mejoras en el encapsulamiento..... 13
Mejorando las pruebas..... 14
Un resultado posible del proceso..... 16
Las pruebas..... 16
El código de la solución..... 17
Más características de Ruby..... 18
self..... 18
if como modificador..... 19
Métodos privados..... 19
Colecciones..... 20
Algunas recomendaciones..... 20
Mantener la consistencia de los protocolos..... 20
Favorecer la declaratividad..... 21
No implementar funcionalidad no requerida..... 23
No hacer suposiciones sobre el dominio..... 23
Reducir la complejidad accidental..... 23
Para cerrar..... 24
Apéndice: convenciones de estilo en Ruby..... 25

Introducción a Ruby (y un repaso de otros temas)

Antes de entrar en los temas específicos de la materia, necesitamos familiarizarnos con Ruby, el lenguaje que vamos a usar durante todo el primer bloque de la cursada.

La forma que elegimos para hacerlo es a través de un ejemplo práctico: vamos a implementar parte de un juego de estrategia, usando la técnica de TDD[1]. A lo largo del ejemplo vamos a ir descubriendo la sintaxis de Ruby, repasando conceptos de diseño con objetos, y reflexionando sobre las decisiones que tomamos al programar.

Como este es un documento estático, no podemos tener ningún "ida y vuelta" con respecto al desarrollo del ejercicio y de las cuestiones teóricas. En ese sentido, es importante que *cuestiones* todo lo que te decimos (en este documento, y durante la cursada en general).

A pesar de que está bien reconocer la experiencia que tenemos como docentes y nuestra preocupación por no decir cosas "porque sí", es importante siempre poder *dudar*, exigir justificaciones hasta que realmente te convenzas de lo que pensás (o hasta que cambies de idea). Esto es lo que va a habilitar, por un lado, el desarrollo de un conocimiento más profundo de los temas (no quedarse con ninguna duda) y, por otro lado, la posibilidad de que el conocimiento avance por fuera de lo que sabemos (tanto los estudiantes *como los docentes*).

El dominio: un juego de estrategia

La siguiente es una descripción breve del dominio de interés[2]:

> *Se nos pide desarrollar un juego de estrategia, donde tenemos unidades (guerreros) que pueden atacar a otros guerreros y ser atacados. Cuando un guerrero ataca, ocasiona un daño a la otra unidad igual a su fuerza menos la defensa del atacado. Al dañarse una unidad, disminuye su vida.*

Con este punto de partida, vamos a empezar a programar.

Estructura del proyecto

Los proyectos en Ruby que usamos en la materia tienen una estructura simple: una carpeta `lib` donde va el código productivo, y una carpeta `spec` donde van las pruebas automatizadas (o *tests*, como se las denomina habitualmente en inglés). Para escribir pruebas vamos a usar *RSpec*, un framework de testing muy conocido en la comunidad de Ruby.

[1] *Test-Driven Development*, o desarrollo dirigido por pruebas.
[2] Este ejemplo está inspirado en el apunte titulado "Teoría de objetos", por Carlos Lombardi y Fernando Dodino, versión 1.2 (2009).

Paso 1: la primera prueba

Como estamos trabajando con TDD, tenemos que decidir con qué prueba empezamos. Recordemos brevemente el flujo de trabajo:

Cada prueba (o *caso de prueba*) representa una regla del dominio de interés a partir de un ejemplo lo suficientemente significativo como para permitirnos generalizar la regla a partir de él, con las verificaciones necesarias para asegurarnos de que nuestro programa cumple con la regla en ese caso. Estas “reglas del dominio” estarán asociadas a los requisitos funcionales del sistema.

Las pruebas involucran un paso de *generalización*: a partir de un escenario concreto, verificamos un comportamiento general asociado a una regla del dominio. Ahora bien, si la regla es tan general que no resulta apropiado especificarla sólo a partir de un único ejemplo, conviene dividirla en sub-reglas (o sub-casos) y probar cada una por separado.

Como ejemplo de esto, supongamos que tenemos una regla en el dominio que dice que "x no puede ser *menor o igual* a cero". En lugar de escribir una sola prueba, podemos dividirla en dos: una que verifique que "x no puede ser *menor* a cero" y otra que verifique que "x no puede ser *igual* a cero". Si ambas pruebas pasan, tenemos un nivel razonable de seguridad de que nuestro programa implementa correctamente la regla general. Si, en cambio, hubiéramos probado la regla general con una sola prueba —digamos, usando el caso del 0—, que esa prueba pase no nos daría suficiente seguridad, porque el caso de los valores negativos quedaría sin cubrir.

Elegir qué probar

Luego de este repaso sobre las pruebas, pensemos en qué opciones tenemos. Según el enunciado, lo que hacen los guerreros es atacar y ser atacados. Todos los requisitos funcionales giran alrededor de esas acciones, entonces nuestra primera prueba debería estar relacionada con eso.

Ahora bien, hay un abanico de casos posibles dentro de esa funcionalidad. ¿Un guerrero que ataca a otro con defensa? ¿Con defensa igual a cero? ¿Qué pasa si la defensa es mayor que la fuerza? ¿Y si el daño supera la vida restante?

El criterio para elegir la primera prueba tiene que ver con encontrar un caso que sea *suficientemente simple* como para poder empezar de forma iterativa e incremental (no queremos escribir el caso más complejo de entrada), pero a la vez *suficientemente significativo* como para que nos aporte valor (no queremos probar algo trivial como "creo un guerrero y verifico que existe").

Por ejemplo, ¿por qué no empezar con algo como "creo un guerrero y verifico que tiene vida"? Porque ahí no estamos probando ningún *escenario* del dominio, sino que nos estamos concentrando en un aspecto más operativo de la lógica de creación de los objetos. En cambio, si probamos qué pasa cuando un guerrero ataca a otro, sí estamos verificando una regla concreta del dominio.

Un buen escenario de partida puede ser: *un guerrero sin defensa es atacado por otro, y le disminuye la vida*. Al poner la defensa en cero, nos sacamos de encima esa complejidad por ahora, y ya estamos probando un caso funcional significativo.

También hay que pensar en los *casos borde*: ¿qué pasa si la defensa es exactamente igual a la fuerza? ¿Y si es mayor? No necesariamente vamos a empezar por esos casos, pero tenerlos presentes nos ayuda a entender el abanico completo de situaciones y a escribir pruebas más adelante que los cubran.

El nombre de la prueba

Ponerle nombre a la primera prueba es difícil, porque todavía no sabemos bien qué vamos a hacer.

Una estrategia útil es *ponerle un nombre provisorio que sea intencionalmente malo*, tan desprovisto de significado que nos resulte imposible dejarlo así (como “test 001”). De esa manera, nos obligamos a volver después y ponerle un nombre que refleje lo que realmente verifica la prueba, una vez que tengamos más información. Si le ponemos un nombre "más o menos aceptable" de entrada, es probable que nos acostumbremos a él y nunca lo mejoremos. En este documento, sin embargo, no vamos a seguir este consejo, y vamos a incluir el nombre final de manera directa para agilizar la lectura.

Hay un par de criterios que nos pueden ayudar a elegir mejores nombres.

Por un lado, el nombre debe describir *el caso de prueba*, no *los datos de prueba.* Una prueba representa una regla del dominio, más allá de los datos puntuales de ejemplo que usemos para ejemplificarla. Si usáramos "un guerrero con 20 de fuerza ataca a otro con 100 de vida y le quedan 80" como nombre de la prueba, estaríamos acoplándonos demasiado a los datos concretos que estamos utilizando. Lo que queremos comunicar es *la regla del juego*: que cuando no hay defensa, el daño es igual a la fuerza del atacante.

Por otro lado, enumerar los pasos de la prueba no suele ser una estrategia efectiva para llegar a un buen nombre. Con esto nos referimos a que el nombre no debería ser una secuencia imperativa de acciones: "creamos un guerrero, lo hacemos atacar, verificamos

que la vida bajó". Suele ser mucho mejor describir de forma *declarativa* qué caso se está verificando.

A su vez, el nombre de la prueba debería sintetizar las tres partes de la estructura de la prueba: en qué contexto estamos (preparación), qué acción se realiza (ejercitación) y qué esperamos que ocurra (verificación). Por ejemplo, un nombre como "agrego un producto" solamente expresa la ejercitación, pero no dice nada sobre el contexto ni sobre lo que se espera; y por lo tanto es un mal nombre.

Por si es que estás olvidado, vamos a repasar ahora estas tres partes que hemos mencionado.

Las tres partes de una prueba

Una prueba tiene tres secciones:

- **Preparación** (conocida también como *setup*, *given* o *arrange*), en donde armamos el contexto del caso que queremos probar.
- **Ejercitación** (conocida como *exercise, when*, o *act*), en donde ejecutamos la acción cuyas consecuencias queremos verificar.
- **Verificación** (conocida como *verify*, *then*, o *assert*), en donde verificamos que las consecuencias de la ejercitación sean las esperadas.

Es útil *separar visualmente las tres partes*, dejando una línea en blanco entre cada una. Si en alguna prueba nos cuesta mantener esta secuencia, probablemente convenga dividirla en dos pruebas separadas.

Para profundizar sobre la estructura de las pruebas, podés consultar:

- *Arrange Act Assert* (http://wiki.c2.com/?ArrangeActAssert): patrón documentado en la wiki de C2, la comunidad histórica de Ward Cunningham sobre patrones de diseño y prácticas de desarrollo.
- *Four Phase Test* (http://xunitpatterns.com/Four%20Phase%20Test.html): del sitio xUnit Test Patterns de Gerard Meszaros, una referencia clásica sobre patrones para pruebas automatizadas.

Escribiendo la prueba

Empecemos entonces creando el archivo de pruebas. En RSpec, la estructura básica es:

```
describe "parte del sistema que estamos probando" do
  specify "nombre de la prueba" do
    # cuerpo de la prueba
  end
end
```

describe abre un grupo de pruebas, y specify define una prueba individual. En lugar de specify, también puede usarse it (de hecho, esto último es lo más popular en la comunidad de Ruby)[3]. El string que le pasamos como argumento a specify es el nombre de la prueba, es decir, la descripción del caso que estamos probando.

Escribamos entonces nuestra primera prueba[4]. Queremos que, después de que un guerrero ataque a otro sin defensa, la vida del atacado haya disminuido. Luego de escribir la verificación y la ejercitación, nos queda así:

```
describe "guerreros" do
  specify "cuando un guerrero ataca a otro que no tiene defensa, el daño es
igual a la fuerza del atacante" do
    guerrero_atacante = ?     # fuerza = 20
    guerrero_atacado = ?# defensa = 0, vida = 100

    guerrero_atacante.atacar_a(guerrero_atacado)

    expect(guerrero_atacado.vida).to eq(80)
  end
end
```

Sabemos que hay detalles de la sintaxis de Ruby que probablemente aún desconocés, pero creemos que podés ir deduciendo qué significa cada cosa (más adelante vamos a hablar con más detenimiento sobre algunas de estas cuestiones).

Prestá atención al nombre de la prueba, en el que no hemos mencionado los datos particulares que estamos usando (ej. 20 de fuerza, 100 de vida), sino que describimos el caso que estamos verificando. Insistimos con esto: lo que queremos comunicar es una regla del juego.

Ahora tenemos que decidir de dónde vamos a obtener los objetos que representan a los guerreros. Suena razonable que sean instancias de una clase. En Ruby, las clases son objetos. Para instanciarlas, les podemos mandar el mensaje new:

```
Guerrero.new(20, 0, 100)
```

Podríamos decidir también que se van a usar parámetros con palabras clave (donde se vincula cada argumento con su parámetro por nombre, en lugar de hacerlo por posición), para que quede más claro el rol de cada uno:

```
Guerrero.new(fuerza: 20, defensa: 0, vida: 100)
```

[3] Hemos elegido usar specify en este apunte porque nos parece más claro (it está pensado para que el nombre de la prueba se lea como una oración en inglés). Sin embargo, es posible que te encuentres con código de ejemplo que use it. Sentite libre de usar la opción que prefieras.
[4] Una estrategia útil para escribir la prueba es empezar por la verificación y de ahí ir completando las otras partes.

Esto no es algo “especial” de la creación de instancias, podemos decidir en general si un mensaje deberá ser enviado usando argumentos nombrados o pasandolos de manera posicional. El mensaje `new` es un mensaje como cualquier otro.

Terminamos así de escribir la primera prueba que, por supuesto, va a fallar:

```
describe "guerreros" do
  specify "cuando un guerrero ataca a otro que no tiene defensa, el daño es
igual a la fuerza del atacante" do
    guerrero_atacante = Guerrero.new(fuerza: 20, defensa: 0, vida: 100)
    guerrero_atacado = Guerrero.new(fuerza: 0, defensa: 0, vida: 100)

    guerrero_atacante.atacar_a(guerrero_atacado)

    expect(guerrero_atacado.vida).to eq(80)
  end
end
```

Analizando nuestras decisiones de diseño

Al escribir esta prueba, quizás sin darnos cuenta, hemos tomado un montón de decisiones de diseño:

- Que los guerreros van a estar representados por instancias de una clase `Guerrero`.
- Que la fuerza, la defensa y la vida son números.
- Que los guerreros entienden el mensaje `atacar_a`, que recibe otro guerrero.
- Que los guerreros entienden el mensaje `vida` (que retorna un número).
- Cómo se crean los guerreros (ej. qué parámetros recibe el constructor).

Está bueno ser conscientes de esto: *cada vez que escribimos una prueba, estamos tomando decisiones de diseño*. Si las tomamos “en automático”, sin darnos cuenta, no nos preguntamos qué alternativas hay. Y hacerse esa pregunta (¿qué otras alternativas tenemos?) es una herramienta fundamental para aprender a diseñar y para llegar a mejores soluciones.

Al margen: ¿cuándo creamos una clase?

Una de las decisiones que tomamos fue que haya una clase `Guerrero`. Ahora, pensemos un momento: ¿cuándo tiene sentido crear una clase nueva? ¿Cómo podemos justificarlo?

Si lo pensamos superficialmente, la respuesta podría parecer obvia, quizás nos imaginamos algo como: “siempre que tengo una nueva entidad en mi dominio, creo una clase”. Pero en realidad no es tan simple. Por ejemplo, si estamos trabajando en el contexto de un sistema para un supermercado y aparece un nuevo producto (digamos,

un detergente), ¿deberíamos crear una clase nueva `Detergente`, o sería sólo una instancia de la clase `Producto`? Otro ejemplo, volviendo al juego de estrategia: si aparecieron conceptos como "ataque", "defensa" o "vida", ¿por qué no creamos clases para ellos?

No nos vamos a detener demasiado sobre esto en este momento. Por ahora basta decir que, en este caso, crear una clase tiene sentido porque ya sabemos que vamos a necesitar al menos dos guerreros, y los guerreros necesitan un protocolo específico (entender mensajes como `atacar_a`, `vida`, etc.) que ningún objeto existente nos provee.

Pero la pregunta queda abierta como algo que vamos a seguir pensando en el futuro…

Algunas características de Ruby

Antes de hacer pasar la prueba (el paso 2 de TDD), vamos a ver algunas características de Ruby. Muchas cosas van a resultar familiares, y otras van a ser bastante diferentes. Vamos a ir señalando las diferencias más importantes, en particular con respecto a Java. Además, el modelo de objetos de Ruby está profundamente influenciado por Smalltalk, que será otro lenguaje que vamos a mencionar.

Todo es un objeto

En Ruby, toda expresión representa un objeto. Los números son objetos, los strings son objetos, `nil` es un objeto, `true` y `false` son objetos. Esto es una diferencia con respecto a Java, donde muchas cosas no se representan con objetos. Por ejemplo, en Java los tipos primitivos (`int`, `boolean`, `double`, etc.) no son objetos, y por ende para operar con ellos no usamos mensajes; las clases no son objetos[5], y por ende necesitamos un operador prefijo especial `new` para poder instanciarlas, etc. En Ruby no existe esa distinción: *todo es un objeto, sin excepción*.

Esto implica, por ejemplo, que `1 + 1` es en realidad un envío de mensaje: le estamos mandando el mensaje `+` al objeto `1`, pasándole `1` como argumento. Es equivalente a escribir `1.+(1)`.

Podemos verificar esto en `irb`, la consola interactiva (el REPL) de Ruby:

```
irb> 1 + 1        # => 2
irb> 1.+(1)       # => 2
```

[5] Más estrictamente, *los nombres de las clases no denotan objetos*, aunque sí hay objetos que representan a las clases (`String` vs. `String.class`). Vamos a entrar más en detalle sobre esto más adelante, cuando exploremos el metamodelo de Java.

Clases y “constructores”

Una clase se define usando la siguiente sintaxis:

```
class Guerrero

end
```

Las clases usan *PascalCase* para sus nombres (primera letra de cada palabra en mayúscula). Las variables y los nombres de mensajes usan *snake_case* (palabras en minúscula, separadas por guiones bajos).

Para crear instancias, le enviamos el mensaje `new` a la clase `Guerrero`. El método que implementa el mensaje `new` en las clases tiene una definición aproximada[6] a la siguiente:

```
def new(...)
  nueva_instancia = self.allocate()
  nueva_instancia.initialize(...)
  return nueva_instancia
end
```

Donde `self` es la clase (el objeto que recibe el mensaje `new`), `allocate` crea una nueva instancia sin inicializar, y los `...` representan todos los parámetros de `new`. Por lo tanto, la inicialización de los guerreros se especificará en un método de instancia llamado `initialize`. Si usamos argumentos posicionales, quedaría así:

```
class Guerrero
  def initialize(fuerza, defensa, vida)
    # ...
  end
end
```

Si, en cambio, queremos usar argumentos nombrados (tipo *keyword*), hay que usar dos puntos y queda así:

```
class Guerrero
  def initialize(fuerza:, defensa:, vida:)
    # ...
  end
end
```

[6] Para quienes tengan curiosidad: la definición real debería usar `send(:initialize)`, porque el mensaje `initialize` es privado. Además, el retorno debería quedar implícito, por convención (mencionaremos esto último más adelante).

`initialize` es un método *de instancia, privado*. Por otro lado, `new` es un método de clase, público. Esto quiere decir que nunca mandamos el mensaje `initialize` de manera directa desde afuera; lo que hacemos es enviarle el mensaje `new` a la clase, quien internamente le enviará `initialize` a la instancia nueva.

Esta separación es una consecuencia de hacer que todo se resuelva mediante el envío de mensajes, en lugar de tener un operador sintáctico particular para crear instancias, y que "el constructor" sea una construcción especial compartida entre el lado de clase y el lado de instancia.

En este sentido, lo que hace Ruby es análogo a lo que sucede en Smalltalk. La palabra "constructor" en estos lenguajes no tiene un significado preciso, por dos razones:

- Primero, porque tanto `new` como `initialize` son mensajes comunes y corrientes, no usan una sintaxis especial ni son palabras reservadas. El lenguaje no necesita definir una construcción dedicada de "constructor".
- Segundo, porque hay *dos* métodos involucrados: un método de clase para crear la instancia, y un método de instancia para inicializarla; y no queda claro a cuál nos referimos cuando decimos "constructor".

A pesar de esta ambigüedad, la palabra "constructor" va a aparecer durante la cursada. En Ruby, cuando decimos "constructor", en general nos referimos al método `initialize`[7].

Variables de instancia

Las variables de instancia se identifican con el prefijo `@`. No es necesario declararlas previamente: simplemente se empiezan a usar y existen a partir de ese momento[8]. En el ejemplo, `@fuerza`, `@defensa` y `@vida` pueden ser variables de instancia que se inicializan en el constructor:

```
class Guerrero
  def initialize(fuerza:, defensa:, vida:)
    @fuerza = fuerza
    @defensa = defensa
    @vida = vida
  end
end
```

Las variables de instancia en Ruby son *privadas al objeto* (no se puede acceder a ellas desde afuera, ni siquiera desde otra instancia de la misma clase). Esta es una diferencia

[7] En Smalltalk, en cambio, con "constructor" normalmente nos referimos a los métodos de creación de instancia (el método `new`, en este caso).

[8] Como comparación, en Java las variables de instancia (también llamadas "campos" o "atributos") deben declararse explícitamente en la clase, junto con su tipo.

con respecto a Java, donde el encapsulamiento es a nivel de la clase: un método de `Guerrero` en Java podría acceder a los atributos `private` de *otro* objeto `Guerrero`.

Si queremos que alguien pueda consultar la vida de un guerrero desde afuera, tenemos que definir un método:

```
def vida()
  return @vida
end
```

`guerrero.vida` es equivalente a `guerrero.vida()`, porque los paréntesis son opcionales en Ruby cuando no hay argumentos. De hecho, *nunca usar paréntesis cuando no hay argumentos* es una convención en Ruby.

Además, un método retorna el valor de su última expresión, así que no hace falta escribir `return`. Se puede usar `return` de forma explícita, pero la convención es no usarlo cuando no es necesario (es decir, cuando el retorno es la última línea del método).

Por lo tanto, una versión equivalente (y preferible) al método anterior puede ser:

```
def vida
  @vida
end
```

El método `vida` es un ejemplo de *getter*: un método cuyo único propósito es retornar el valor de una variable de instancia. `guerrero.vida` es una colaboración en la que se envía el mensaje `vida`, no estamos accediendo a la variable de instancia directamente.

También podríamos definir un *setter*, de la siguiente manera:

```
def vida=(nueva_vida)
  @vida = nueva_vida
end
```

Esto nos permitiría que los guerreros entiendan el mensaje `vida=`:

```
un_guerrero.vida=(20)
```

O, equivalentemente (y más idiomático):

```
un_guerrero.vida = 20
```

Como definir *getters* a mano es muy común, Ruby provee un atajo:

```
class Guerrero
  attr_reader :vida, :defensa
```

```
    # ...
end
```

`attr_reader :vida` define automáticamente un método `vida` que retorna el valor de `@vida`. Si en el futuro necesitamos que el método haga algo más complejo, eliminaríamos el `attr_reader` y definiríamos el método a mano, sin necesidad de cambiar nada en el código que envía el mensaje. Existen también `attr_writer` (que define solo el *setter* `vida=`) y `attr_accessor`[9] (que define *getter* y *setter*).

⚠️ No está de más decir que definir *getters* y (especialmente) *setters* suele ser una práctica que va completamente en contra de la preservación del encapsulamiento, por lo que es algo que vamos a evitar.

Símbolos

La expresión `:vida` se conoce como *símbolo literal*, y denota un objeto de tipo *símbolo* (instancia de la clase `Symbol`). Los símbolos son similares a los strings, pero en lugar de ser su función representar "un cacho de texto", representan un *identificador*. Una característica de los símbolos es que son inmutables, y todos los símbolos con el mismo nombre tienen la misma identidad.

`nil`

Un método vacío, o cuya última expresión evaluada sea `nil`, retorna `nil`. Este es el equivalente de `null` en Java, pero en Ruby `nil` es un objeto (instancia de `NilClass`), y puede responder mensajes. En cambio, en Java, `null` no es un objeto (justamente, es la ausencia de un objeto), y si intentamos enviarle un mensaje, va a explotar con una `NullPointerException`.

❦

Con estas herramientas del lenguaje, ya estamos en condiciones de seguir con el ciclo de TDD y hacer pasar nuestra primera prueba.

[9] En el vocabulario de Smalltalk, los getters y setters se conocen como "accessors". Ruby toma prestado estas palabras de ahí.

Paso 2: haciendo pasar la prueba

Un enfoque posible para hacer pasar la prueba rápidamente es que, dentro de `atacar_a`, el atacante manipule directamente la vida del atacado:

```
def atacar_a(otro_guerrero)
  otro_guerrero.vida = otro_guerrero.vida - @fuerza
end
```

Esto funciona, pero tiene un problema conceptual: *estamos rompiendo el encapsulamiento* del otro guerrero. En lugar de decirle que haga algo, le estamos pidiendo su vida, haciendo una operación nosotros, y finalmente modificándole la vida de acuerdo con el resultado. Es como si en vez de pedirle a alguien que nos haga un café, le agarráramos el brazo y lo moviéramos nosotros para hacer el café.

Pero bueno, por ahora hacemos eso, y ya es suficiente para que la prueba pase. El código mínimo quedaría así:

```
class Guerrero
  attr_accessor :vida

  def initialize(fuerza:, defensa:, vida:)
    @fuerza = fuerza
    @vida = vida
  end

  def atacar_a(otro_guerrero)
    otro_guerrero.vida = otro_guerrero.vida - @fuerza
  end
end
```

Paso 3: refactorizar

Vamos a ver ahora qué mejoras podemos hacer en la estructura de nuestro programa, sin afectar su comportamiento (y, en consecuencia, manteniendo la prueba pasando).

Mejoras en el encapsulamiento

Para recuperar el encapsulamiento, la solución es “pasarle la pelota” al otro guerrero: que el guerrero atacante le diga al atacado que reciba daño, y que el atacado se encargue de procesar ese daño internamente.

```
class Guerrero
  attr_reader :vida

  def initialize(fuerza:, defensa:, vida:)
    @fuerza = fuerza
    @vida = vida
  end

  def atacar_a(otro_guerrero)
    otro_guerrero.recibir_daño(@fuerza)
  end

  def recibir_daño(cantidad_de_daño)
    @vida -= cantidad_de_daño
  end
end
```

Noten que con este cambio el *setter* `vida=` ya no se necesita, y podemos eliminarlo (ahora usamos `attr_reader` en lugar de `attr_accessor`). Esta es una mejora muy buena, porque no queremos que se pueda "pisar" la vida desde afuera con un valor arbitrario; la vida solamente debería modificarse indirectamente (y de manera controlada) como resultado de otras acciones (como recibir daño).

Mejorando las pruebas

Un problema que aparece al escribir varias pruebas es que en cada una tenemos que crear guerreros pasando argumentos para todos los parámetros, incluso los que no son relevantes para la regla que queremos especificar con la prueba. Por ejemplo, si queremos probar que el ataque funciona, nos importa la fuerza del atacante y la vida del atacado, pero no nos importa la defensa del atacante ni la fuerza del atacado.

Una solución puede ser extraer un *factory method* dentro de las pruebas: un método auxiliar que nos permite crear guerreros proporcionando solamente lo relevante, y usando valores predeterminados para el resto. Quedaría así:

```
describe "guerreros" do
  specify "cuando un guerrero ataca a otro que no tiene defensa, el daño es
igual a la fuerza del atacante" do
    guerrero_atacante = un_guerrero_con(fuerza: 20)
    guerrero_atacado = un_guerrero_con(defensa: 0, vida: 100)

    guerrero_atacante.atacar_a(guerrero_atacado)
```

```
    expect(guerrero_atacado.vida).to eq(80)
  end

  def un_guerrero_con(fuerza: 0, defensa: 0, vida: 100)
    Guerrero.new(fuerza: fuerza, defensa: defensa, vida: vida)
  end
end
```

Ahora la prueba se lee de manera más clara: vemos que nos importa que el atacante tenga 20 de fuerza, que el atacado tenga 0 de defensa y 100 de vida, y nada más. Los demás valores son irrelevantes para este caso, y eso queda reflejado en el código.

📝 Normalmente habríamos esperado tener un par de pruebas más para hacer este cambio, pero lo hacemos aquí ahora por una cuestión de extensión del documento.

Las pruebas forman parte del modelo que estamos construyendo al programar. Es decir, debemos tratar de lograr que el código de las pruebas refleje de la manera más clara posible lo que está modelando: una especificación sobre cómo se tiene que comportar el sistema en un caso concreto, de acuerdo al dominio de interés.

Si la prueba está llena de datos irrelevantes (aunque necesarios para su ejecución), es más difícil entender qué es lo que se está verificando. La contracara de esto es que *queremos poder ver de manera explícita y clara* todo aquello que sea relevante para el caso, y que eso no quede oculto.

Por ejemplo, el siguiente sería un contraejemplo:

```
specify "cuando un guerrero ataca a otro que no tiene defensa,
el daño es igual a la fuerza del atacante" do
  guerrero_atacante = un_guerrero_con(fuerza: 20)
  guerrero_atacado = un_guerrero_con()

  guerrero_atacante.atacar_a(guerrero_atacado)

  expect(guerrero_atacado.vida).to eq(80)
end
```

A pesar de que la prueba sigue pasando, se entiende bastante menos lo que está sucediendo.

Un resultado posible del proceso

Veamos cómo podría quedar un resultado posible, luego de haber seguido el proceso de TDD, y habiendo consultado por las reglas del juego correspondientes a algunos casos especiales que no habían sido especificados.

> 💭 Antes de seguir leyendo, es buena idea hacer el ejercicio de pensar en qué otros casos de prueba podría haber. Intentá elaborar una lista concreta de pruebas. ¿Qué casos borde habría que tomar en cuenta? ¿Qué situaciones no fueron especificadas explícitamente como parte de la descripción del dominio (y por las cuales tendríamos que consultar)?

El ejemplo muestra cómo levantar una excepción, cómo verificar que esperamos que se haya levantado una excepción desde una prueba, y algunos otros detalles de los que no hemos hablado aún.

Las pruebas

```
require "guerrero"

describe "guerreros" do
  specify "cuando un guerrero ataca a otro que no tiene defensa, el daño es igual a
la fuerza del atacante" do
    guerrero_atacante = un_guerrero_con(fuerza: 20)
    guerrero_atacado = un_guerrero_con(defensa: 0, vida: 100)

    guerrero_atacante.atacar_a(guerrero_atacado)

    expect(guerrero_atacado.vida).to eq(80)
  end

  specify "cuando un guerrero ataca a otro que tiene defensa, el daño se reduce de
acuerdo a la defensa del atacado" do
    guerrero_atacante = un_guerrero_con(fuerza: 20)
    guerrero_atacado = un_guerrero_con(defensa: 10, vida: 100)

    guerrero_atacante.atacar_a(guerrero_atacado)

    expect(guerrero_atacado.vida).to eq(90)
  end

  specify "si la defensa del atacado es mayor a la fuerza del atacante, el ataque
no causa daño" do
    guerrero_atacante = un_guerrero_con(fuerza: 20)
    guerrero_atacado = un_guerrero_con(defensa: 100, vida: 100)

    guerrero_atacante.atacar_a(guerrero_atacado)
```

```
    expect(guerrero_atacado.vida).to eq(100)
  end

  specify "el nivel mínimo de vida es cero" do
    guerrero_atacante = un_guerrero_con(fuerza: 50)
    guerrero_atacado = un_guerrero_con(defensa: 0, vida: 30)

    guerrero_atacante.atacar_a(guerrero_atacado)

    expect(guerrero_atacado.vida).to eq(0)
  end

  specify "un guerrero no puede atacarse a sí mismo" do
    guerrero = un_guerrero_con(fuerza: 20, vida: 100)

    expect do
      guerrero.atacar_a(guerrero)
    end.to raise_error("un guerrero no puede atacarse a sí mismo")
  end

  def un_guerrero_con(fuerza: 0, defensa: 0, vida: 100)
    Guerrero.new(fuerza: fuerza, defensa: defensa, vida: vida)
  end
end
```

El código de la solución

```
class Guerrero
  attr_reader :vida
  attr_reader :defensa

  def initialize(fuerza:, defensa:, vida:)
    @fuerza = fuerza
    @defensa = defensa
    @vida = vida
  end

  def atacar_a(otro_guerrero)
    raise "un guerrero no puede atacarse a sí mismo" if self == otro_guerrero

    otro_guerrero.recibir_daño(@fuerza)
  end
```

```
  def recibir_daño(cantidad_de_daño)
    daño_final = (cantidad_de_daño - @defensa).clamp(0, @vida)
    @vida -= daño_final
  end
end
```

Más características de Ruby

self

En Ruby, la manera de referirnos al objeto receptor del mensaje (es decir, el objeto que se está haciendo cargo de la ejecución) es con la palabra reservada `self`. Esto es equivalente a `this` en Java.

Al margen: `self` vs. `this`

Ruby toma el nombre `self` a partir de Smalltalk, que usa la misma palabra reservada. Por otro lado, Java tomó el `this` de C++. Esto es un detalle, pero el uso de estas palabras refleja algunas ideas profundas con respecto a cómo estos lenguajes consideran a los objetos.

En la tradición de Smalltalk, los objetos se piensan como *entidades activas*, agentes computacionales que pueden referirse a sí mismos a medida que actúan (y de ahí el uso de `self`, o "sí mismo" en inglés). La palabra `self`, además, tiene sentido conceptual en cualquier contexto de ejecución (dicho informalmente, siempre hay "un alguien" ejecutando).

En cambio, la tradición de C++ ve a los objetos como una estructura de datos. `this` es un puntero a "esta estructura", la que estoy manipulando ahora. No es casual que, en Java, `this` no exista en el contexto de los métodos estáticos, porque ahí no hay un objeto que estemos manipulando, es simplemente un procedimiento.

Por otro lado, cuando enviamos un mensaje sin receptor explícito, el receptor será `self` (o sea, el mismo objeto que recibió el mensaje original).

Por ejemplo, el método `recibir_daño` podría reescribirse usando mensajes para obtener la defensa y la vida:

```
def recibir_daño(cantidad_de_daño)
  daño_final = (cantidad_de_daño - defensa).
    clamp(0, vida)
  # ...
end
```

Y esto es equivalente (explicitando los receptores y poniendo los paréntesis opcionales) a:

```
def recibir_daño(cantidad_de_daño)
  daño_final = (cantidad_de_daño - self.defensa()).
    clamp(0, self.vida())
  # ...
end
```

O sea, el guerrero se manda a sí mismo los mensajes `defensa` y `vida`.

`if` como modificador

En el método `atacar_a` usamos una forma sintáctica particular del `if` que Ruby permite cuando el cuerpo es una sola expresión:

```
raise "..." if self == otro_guerrero
```

Esto es equivalente a:

```
if self == otro_guerrero
  raise "un guerrero no puede atacarse a sí mismo"
end
```

Métodos privados

En Ruby, los métodos son públicos por defecto. Para marcar métodos como privados, usamos la palabra `private`. La convención es poner `private` una sola vez. Todos los métodos definidos debajo de esa línea pasan a ser privados.

En el siguiente ejemplo, el método `m1` es público, y los métodos `m2` y `m3` son privados:

```
class UnaClase
  def m1
    # ...
  end

  private

  def m2
    # ...
  end
```

```
    def m3
      # ...
    end
  end
```

El único receptor válido de un mensaje que resulte en la ejecución de un método privado es `self` (¿por qué?).

Colecciones

Las dos colecciones más usadas en Ruby son:

- Las listas (llamadas *arrays*), que pueden crearse usando corchetes, por ejemplo: `[1, 2, 3]`
- Los diccionarios (llamados *hashes*), que pueden crearse usando llaves, por ejemplo: `{ 'clave' => 'valor', 1 => 2 }`

Para tener más información sobre estas colecciones, podés buscar la clase `Array` y `Hash` en la documentación oficial del lenguaje. Otras colecciones usadas son los conjuntos (clase `Set`) y los rangos (clase `Range`).

Algunas recomendaciones

Para cerrar este documento, recopilamos algunos consejos y recomendaciones sobre diseño que van a ser relevantes a lo largo de toda la cursada. Si bien muchas de ellas aplican a cualquier lenguaje, las ilustramos con ejemplos en Ruby.

Mantener la consistencia de los protocolos

Aun cuando Ruby es un lenguaje dinámicamente tipado, es importante que *el protocolo*[10] de nuestros objetos sea coherente. Esto incluye tanto los mensajes que entiende un objeto como lo que retorna cada uno de ellos.

Por ejemplo, si un guerrero tiene un método `vida` que retorna un número, debería retornar *siempre* un número. Sería incorrecto, por ejemplo, que retorne el string `"sin vida"` cuando la vida llega a cero: eso obligaría a quien envía el mensaje a contemplar dos tipos de respuesta[11] posibles, complicando innecesariamente nuestro programa.

[10] Es decir, la interfaz del objeto junto con el significado y las expectativas que tenemos con respecto a cada mensaje que forma parte de ella.
[11] En este contexto, con “tipo” nos referimos al protocolo que esperamos que un objeto respete. En el ejemplo, sería un problema cada vez que queramos enviarle mensajes exclusivos del protocolo de los números (o de string) al objeto retornado.

Favorecer la declaratividad

Un criterio que aparecerá recurrentemente en la materia es la preferencia por código *declarativo* (que describe *qué* queremos) frente a código *imperativo* (que describe *cómo* lo hacemos). Si bien esto vale para cualquier lenguaje, Ruby ofrece herramientas que facilitan escribir código más declarativo.

Veamos algunos ejemplos.

Acceder a elementos de una colección

Cuando queremos el primer elemento de una lista, preferimos enviar el mensaje `first` en lugar de acceder por índice.

Por ejemplo, si tenemos una lista de enemigos:

```
# Más imperativo
enemigos[0]

# Más declarativo
enemigos.first
```

La segunda versión expresa la *intención* (obtener el primero) en lugar del *mecanismo* (acceder a la posición cero).

Iteración

En lugar de usar `for` o `while`, buscamos formas más declarativas. Por ejemplo, si queremos agregar 10 guerreros a una colección:

```
# Bastante imperativo
i = 1
while i ≤ 10
 @guerreros.push(crear_guerrero)
 i += 1
end

# Un poco menos imperativo
for i in 1..10 do
  @guerreros.push(crear_guerrero)
end
```

```
# Menos imperativo
10.times do
  @guerreros.push(crear_guerrero)
end
```

Dejar que los objetos decidan por su cuenta cómo comportarse

En lo posible, evitamos usar `if` cuando los objetos pueden tomar las decisiones por su cuenta[12].

Un ejemplo de esto es cuando usamos un hash con valor predeterminado en lugar de verificar con un `if` si una clave existe. Digamos que un guerrero lleva la cuenta de cuántas veces venció a cada enemigo, entonces:

```
# Con if
def initialize
  @victorias = {}
end

def registrar_victoria_contra(enemigo)
  if @victorias.key?(enemigo)
    @victorias[enemigo] += 1
  else
    @victorias[enemigo] = 1
  end
end

# Con Hash.new(0), que retorna 0 para claves que no existen
def initialize
  @victorias = Hash.new(0)
end

def registrar_victoria_contra(enemigo)
  @victorias[enemigo] += 1
end
```

[12] Generalmente (aunque no necesariamente) esto ocurre cuando usamos polimorfismo. El ejemplo que damos aquí es el de un caso *que no involucra polimorfismo*.

No implementar funcionalidad no requerida

Un consejo, tanto para las pruebas como para el código productivo, es *no implementar funcionalidad que no fue pedida*. Esto a veces se conoce como *gold plating* (baño de oro): agregar cosas que nadie pidió, creyendo que "quizás podrían servir".

Además de ser más trabajo, probablemente estamos introduciendo complejidad adicional, haciendo algo que no se usa, e incluso que potencialmente va en contra de lo que se quiere en el negocio. Por ejemplo, si no se pidió que cada guerrero tenga un nombre propio, esto no tendría que aparecer en nuestro programa.

Este criterio también se aplica a las pruebas: no tiene sentido escribir pruebas para funcionalidad que no existe ni fue solicitada. Las pruebas especifican comportamiento esperado del sistema, y ese comportamiento debería estar motivado por los requisitos.

No hacer suposiciones sobre el dominio

Esperamos que consultes sobre los casos no especificados respecto al dominio de interés. Podés hacer suposiciones temporales, pero tenés que tener presente que todas ellas deberán ser validadas consultando con los docentes.

Reducir la complejidad accidental

En *"No Silver Bullet"* (1986), Fred Brooks plantea la distinción entre dos tipos de complejidad en un programa: la *complejidad esencial*, que es inherente al dominio del problema que estamos resolviendo, y la *complejidad accidental* (o complicaciones innecesarias), que es la que introducimos como consecuencia de nuestra manera de resolver el problema. Es decir, la complejidad esencial es la que hace que el problema sea lo que es, mientras que la accidental es la que nosotros agregamos y que, en principio, podríamos evitar. Entonces, para minimizar la complejidad total de nuestro programa, tenemos que apuntar a minimizar la complejidad accidental (ya que la complejidad esencial no depende de nosotros).

¿Cómo podemos hacerlo concretamente? No hay una única manera, pero podemos mencionar algunas. Por ejemplo, desde el diseño, evitando depender de cuestiones de implementación que no tienen que ver con el problema que estamos resolviendo, y eliminando patrones de colaboraciones repetidas (extrayendo métodos, delegando responsabilidades, etc.). Desde la escritura del código, manteniendo la prolijidad: seguir las convenciones definidas, no dejar código comentado ni comentarios superfluos que distraigan a quienes leen el programa, etc. Y desde el proceso de trabajo, por ejemplo, escribiendo buenos mensajes de commit que representen fielmente el cambio que hicimos.

Para cerrar

A lo largo de este documento hicimos varias cosas a la vez: nos familiarizamos con Ruby, repasamos el proceso de TDD (con un ejemplo de una primera iteración), y reflexionamos sobre las decisiones de diseño que tomamos al programar.

También fuimos señalando algunas diferencias y similitudes entre Ruby y otros lenguajes que ya conocemos, como Java y (quizás) Smalltalk. A veces, estas diferencias no tienen mucha importancia (por ejemplo, las convenciones sobre usar `camelCase` o `snake_case`). Otras veces, responden a ideas más profundas, como distintas concepciones sobre qué es un objeto, cómo se organiza un programa y cuál es el rol de quien programa. A medida que avancemos en la cursada, vamos a seguir encontrando este tipo de contrastes entre los distintos lenguajes, y la idea es que nos sirvan para entender mejor los distintos enfoques que conviven hoy en día en la programación orientada a objetos.

Por ahora trabajamos con un modelo bastante tradicional, en el contexto de un ejercicio muy simple. En los próximos temas vamos a empezar a preguntarnos qué pasa cuando este modelo parece no ser suficiente. Mientras tanto, lo más importante es que empieces a experimentar con Ruby. Como mencionamos en la presentación de la materia, no dejar pasar las primeras semanas sin familiarizarse con el lenguaje es fundamental para poder encarar el trabajo práctico con confianza.

Apéndice: convenciones de estilo en Ruby

A lo largo de la cursada seguimos algunas convenciones estándar de Ruby. Las más importantes para arrancar son:

- `snake_case` para nombres de variables, parámetros y métodos: `atacar_a`, `daño_final`, `otro_guerrero`. Viniendo de Java, donde se usa `camelCase`, este es uno de los cambios que hay que incorporar desde el principio.
- `PascalCase` para nombres de clases y módulos: `Guerrero`, `Atacante`.
- Indentación de 2 espacios.
- No usar `return` en la última expresión de un método.
- No usar paréntesis en envíos de mensaje sin argumentos: `guerrero.vida` en lugar de `guerrero.vida()`.
- No usar paréntesis en definiciones de métodos sin parámetros: `def vida` en lugar de `def vida()`.
- Comillas dobles para strings: `"hola"` en lugar de `'hola'`.
    - La convención en la comunidad de Ruby es ligeramente más compleja, pero para simplificar vamos a seguir esta convención en Objetos 3.
- Los métodos que retornan booleanos terminan en `?`: `vivo?`, `puede_atacar?`.
- Preferir usar `unless condición` en lugar de `if !condición`.
- Asignación abreviada para los mensajes binarios: `total += 1` en lugar de `total = total + 1`. Más información sobre esto en la documentación de Ruby.

Usamos *RuboCop* como herramienta de análisis estático para verificar estas convenciones automáticamente.